CODEFRONTIER HACKATHON · ROUND 1 · THE PLAN

Problem Statement 01
Prevent Critical Knowledge
From Leaving With People

Continuum

Project handovers that show their work.

Continuum rebuilds the current state of a project from its repository, team chat, notes and task board, turns it into a handover where every statement points to its source, asks the departing member only the questions the evidence cannot answer, and lets a reviewer approve exactly what the successor is allowed to see.

TEAM
Binary Builders

MEMBERS
Pawan Singh Bhandari
Rahul Bramhankar

SUBMITTED
6 November 2026
Round 1 Proposal (PDF)

AT A GLANCE

The proposal in one page

When someone leaves a small software team, the code stays but the reasons, the half-finished work and the “ask Aditi, she knows” knowledge leave with them. Handover notes are usually written in the last few days, from memory, by the person with the least time left. We think that order is backwards.

What we are solving

The handover of an **active software project** in a **small team (3–25 people)** whose work lives in a Git repository, a team chat, shared notes and a task board — from the day a departure is known until the successor’s first productive week. Primary setting: student tech clubs and college project teams, where turnover happens every semester.

What we will build

A web app that follows the required **Connect → Reconstruct → Verify → Review → Handover** flow and produces a **role-scoped handover**: current state, decisions and their reasons, owners, risks, dependencies, contacts and a first-week plan — with a citation on every line and an honest label wherever evidence is thin, missing or contradictory.

Four ideas that make Continuum different

Idea	What it means in practice
1. Claims, not summaries	The handover is built from atomic claims. Each carries an evidence state — VERIFIED INFERRED CONFLICTING MISSING — and one or more source links. Nothing is presented as fact without a source.
2. Gap Interview	Instead of asking the leaver to write a document from scratch, Continuum asks them a short list of targeted questions only where evidence is missing or contradictory. Their answers become new, citable evidence.
3. Permission before generation	Access rules are applied before any text reaches the AI model. A successor’s view is generated only from sources that person is cleared to see, so the model cannot leak what it was never given.
4. Handover as a loop	The successor acknowledges each section or marks it “unclear”. Unclear items go back to the reviewer and leaver while the leaver is still reachable — the notice period becomes a working window, not a deadline.

Where each judging criterion is answered

Criterion	Points	Sections in this document
Understanding of the problem	25	§2 What is lost and why documentation fails · §3 Users, scenarios and the part we solve
Proposed solution	25	§4 Solution · §5 End-to-end workflow with screens · §6 Feature scope
Technical plan	20	§7 Architecture, stack and evidence model · §8 Hardest technical problems · §9 Trust, privacy and permissions
Originality of approach	15	Ideas 1–4 above, expanded in §4 and §8
Feasibility and scope	15	§6 Scope and exclusions · §10 Team · §11 Three-round plan, cut list and test plan

Also covered: expected impact (§12), future scope (§13), team and participant details with tool disclosures (§14).

§2 · UNDERSTANDING OF THE PROBLEM

Knowledge does not disappear. It was never in one place.

Most teams do have documents. The problem is that the knowledge a successor actually needs is not in them — it is spread across commit messages, PR review threads, late-night chat discussions, meeting notes and the leaver's memory.

What exactly is lost when someone leaves

Type of knowledge	Where it usually lives	What happens without it
Reasons behind decisions	PR review comments, chat threads, a meeting note no one links to	The successor reverses a decision that was made for a good reason, and repeats the same mistake.
True state of unfinished work	Open PRs, stale branches, TODOs, "almost done" tasks	Work marked 90% done turns out to be 40% done; deadlines slip.
Dependencies and access	Environment configs, third-party dashboards, who owns which credential	Production breaks and nobody knows who holds the account.
Recurring problems and workarounds	Chat ("restart the bot again"), issue comments, the leaver's habits	Known issues are rediscovered from scratch, often under pressure.
People and contacts	DMs, email, the leaver's phone	The successor doesn't know who to ask — the sponsor, the faculty coordinator, the API vendor.

Why existing documentation fails

1 It records the what, not the why

A README explains how to run the project. The reason payments moved to UPI links is in a chat thread from August.

2 It drifts out of date

Docs are written once and code keeps changing. The README says "deployed on Render"; the last three commits configure Vercel.

3 The handover note is written at the worst time

In the last week, from memory, by a person with exams, a new job or simply less motivation. It covers what they remember, not what matters.

4 The successor doesn't know what to ask

Unknown unknowns. They discover the gap only when something breaks — usually after the leaver has stopped replying.

Why a plain AI summary is not the answer either

Pointing a language model at a chat export produces fluent text, but it mixes fact with guesswork, cannot say where a statement came from, quietly picks a side when sources disagree, and happily repeats something from a private channel. For a handover — which people will act on — those are exactly the failures that matter. This is why the problem statement insists on traceability, uncertainty marking, human approval and permission awareness, and why we treat those as the core of the product rather than add-ons.

> **How we framed the problem**
>
> A handover has three jobs: **tell the truth** about the project's state (traceable), **tell the successor what to do next** (actionable), and **tell them only what they may know** (permission-aware). These map one-to-one to the "Success Looks Like" criteria in the brief, and every feature in §6 serves at least one of them.

§3 · USERS, SCENARIOS AND SCOPE

Who feels this problem

User	Who they are	What they need from Continuum
Departing member the “leaver”	Graduating club lead, intern finishing a term, volunteer stepping down	Hand over properly without writing a long document. Spend their limited time only on what nobody else can answer.
Incoming member the “successor”	Junior taking over, new hire, replacement volunteer	Understand the current state, owners, risks and next steps quickly — and be able to check any statement against its source.
Reviewer	Team lead, project mentor, faculty coordinator, maintainer	Confidence that the handover is correct and that it shares nothing it shouldn’t. A fast way to correct, reject and approve.
Workspace admin	Club/organisation tech lead	Control which sources are connected and which roles may see which channels, folders and repositories.

Three departure scenarios we design for

Graduating club lead

A final-year student hands the club’s event app to a second-year. Time: two to three weeks before exams. Sources: GitHub, Discord, Google Docs. Sensitive: sponsor contacts, payment keys.

Intern leaving mid-project

A six-month intern leaves a feature half-built. Time: notice period of a week or two. Sources: repo, Slack, task board. Sensitive: client names, internal channels.

Volunteer stepping down

A maintainer of a community or NGO tool rotates out. Often no formal notice. Sources: public repo, issue tracker, chat. Sensitive: donor or member data.

Why we start with student tech teams

- **Turnover is guaranteed and predictable.** Every semester and every graduating batch triggers handovers, so the problem recurs on a schedule.
- **The sources are standard and exportable.** GitHub, Discord or Slack, and Google Docs/Notion all offer APIs or exports — no unrealistic enterprise access is needed.
- **We are the users.** We work in this environment, can test with real club teams, and can build realistic synthetic data without touching confidential company material.
- **It generalises.** The same workflow applies to startups, research groups and volunteer teams; only the connectors and role labels change.

Evidence and validation

Our Round 1 understanding comes from working inside student project teams and from the brief itself, so we treat it as a hypothesis, not proof. Before and during Round 2 we will run **short structured interviews with at least five club leads, mentors or maintainers** (what was lost in their last handover, how long the successor took to become productive, what they would never share with a successor) and a **handover test**: one person takes over a seeded demo project with and without Continuum. Findings and any resulting changes will be reported in the Round 2 document.

> **The specific part of the problem we solve — and what we leave out**
>
> **In scope:** reconstructing project state from authorised sources, evidence-linked claims with uncertainty labels, a gap interview with the leaver, human review and approval, and a role-scoped handover with acknowledgement. **Out of scope for this event:** HR offboarding, company-wide knowledge bases, live meeting transcription, and live OAuth connectors for every chat tool — we use official exports instead (see §6).

§4 · PROPOSED SOLUTION

Continuum: a handover that shows its work

Continuum turns scattered project activity into a reviewed, role-appropriate handover. It does three things a document or a chatbot cannot: it keeps every statement tied to evidence, it asks the leaver only about the gaps, and it filters by permission before generating anything.

The core unit: an evidence-backed claim

All imported material is broken into **evidence units** — one commit, one PR comment, one chat thread, one paragraph of a note. The AI extracts **claims** from them (a decision, an owner, a risk, a dependency, a next action), and every claim carries its evidence state:

State	Meaning	How it is shown to people
VERIFIED	At least one source states it directly, and an automated check confirms the source supports it.	Normal text with source chips.
INFERRED	A reasonable conclusion drawn from sources, but no source says it outright.	Labelled "AI inference — not stated in any source".
CONFLICTING	Two or more sources disagree.	Both versions shown side by side. Never auto-resolved.
MISSING	Something a handover needs has no evidence at all (for example, an open PR with no owner).	Listed as a gap; turned into a Gap Interview question.
CONFIRMED	A human reviewer approved or corrected the claim.	Shows who confirmed it and when.

What the successor receives

1. **Project in five lines** — goal, users, stack, current phase.
2. **Current state by workstream** — done, in progress, blocked, with the real status of open work.
3. **Decision log** — a timeline of key decisions and why they were made.
4. **Your first week** — ordered next actions, each linked to its issue, PR or thread.
5. **Risks and landmines** — recurring problems, workarounds, fragile parts.
6. **Dependencies and people** — services, credential owners, who to ask for what.
7. **Still unresolved** — open questions and conflicts the team could not settle, stated honestly.

The Gap Interview

After reconstruction, Continuum knows what it could not find. It turns each MISSING or CONFLICTING item into a short, specific question for the leaver, for example:

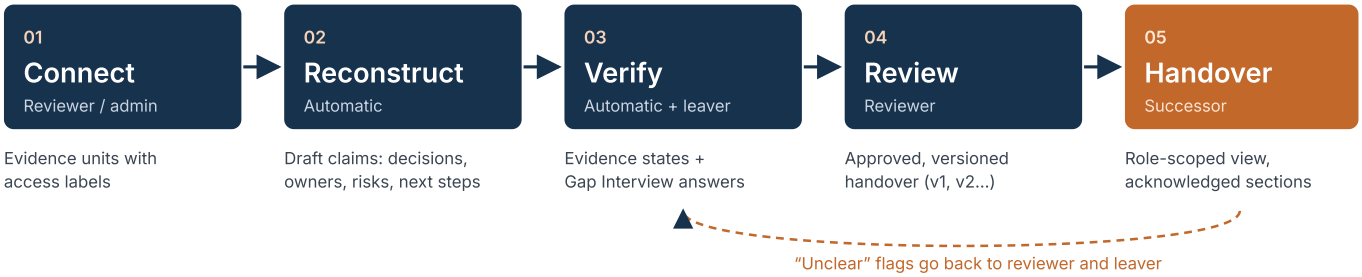
> "PR #91 *'temporary fix for login loop'* was merged with a TODO on 18 Sep. Is this still temporary, and what would the proper fix be?"
>
> "The README says production runs on Render, but commits since 2 Oct configure Vercel. Which is live today?"
>
> "Who owns the college SMTP account used for event emails? No source mentions it."

The leaver answers in a few minutes per question, and each answer is stored as a new evidence unit — cited as `Exit Q&A · Aditi · 3 Nov` — so even their personal knowledge becomes traceable. A typical handover should need a short list of questions, not a blank document.

§5 · COMPLETE PRODUCT WORKFLOW

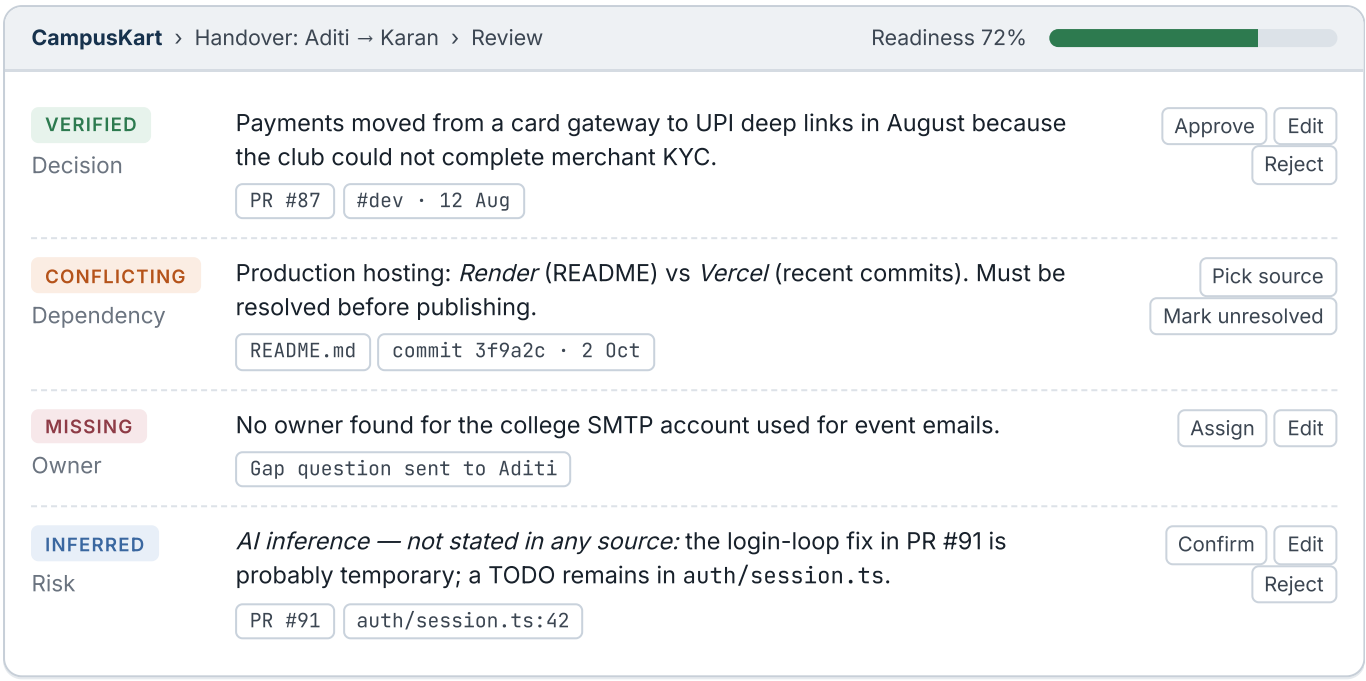
From connected sources to an approved handover

The workflow follows the five stages required by the brief. Each stage has a clear owner and a clear output.



Stage	What the user does	What Continuum does
1 · Connect	Reviewer creates a handover (leaver, successor, successor's role, target date), connects a GitHub repo with a read-only token, and uploads a Slack/Discord export, notes and a task-board export. Chooses which channels and folders are in scope.	Parses everything into evidence units, scans for secrets and personal data, and attaches an access label from the source (e.g. `#core-team → leads`).
2 · Reconstruct	Waits a few minutes; can watch progress per source.	Groups evidence by thread and time window, extracts claims, builds the decision timeline, ownership map, risk list, dependency list and next actions.
3 · Verify	Leaver answers the Gap Interview (link sent by the reviewer).	Checks every citation, assigns evidence states, detects conflicts and gaps, computes a **handover readiness score**, and folds the leaver's answers back in as evidence.
4 · Review	Reviewer goes through the claim ledger: approve, edit, reject, mark sensitive, or redact. Conflicts must be resolved or explicitly marked "unresolved — successor to follow up".	Records every change with author and time, keeps version history, and locks an approved version when published.
5 · Handover	Successor opens their link, reads the "Start here" page, clicks any source chip to see the original, and acknowledges each section or flags it as unclear.	Shows only claims whose evidence the successor's role may see, tracks acknowledgement, and routes unclear flags back to the reviewer and leaver.

What the review screen looks like



Mock-up based on our synthetic demo project "CampusKart". Names and data are fictional.

§6 · MAIN FEATURES AND SCOPE

What we will build, and what we won't

The brief rewards a focused workflow that works end to end over a long feature list. We split features into what must work in Round 2 (the full central workflow), what we will add in Round 3, and what we are deliberately not building.

Feature	Description	Round
Source connectors	GitHub (commits, PRs, issues, review comments, README) via read-only token; Slack and Discord official JSON exports; notes and docs as Markdown/PDF/DOCX; task board via GitHub Issues or Trello CSV.	2
Evidence store with access labels	Every unit keeps author, timestamp, link back to the original, and an access label inherited from its channel, folder or repo.	2
Project-state reconstruction	Decisions with reasons, owners, open work and its real status, risks, recurring problems, dependencies, contacts, next actions, decision timeline.	2
Claim verification	Citation checks and evidence states (Verified / Inferred / Conflicting / Missing). Claims that fail verification are never shown as verified.	2
Gap Interview	Auto-generated questions for the leaver; answers stored as citable evidence.	2
Review and approval	Claim ledger with approve / edit / reject / redact, readiness score, version history, publish lock.	2
Role-scoped handover view	"Start here" page for the successor with source chips that open the original evidence. Two roles in Round 2: Lead and Contributor.	2
Cross-source conflict resolution	Detect contradictions between sources across the whole project (not just within one thread) and resolve them side by side.	3
Acknowledgement loop and version diff	Successor acknowledges or flags each section; flags route back; reviewers see what changed between v1 and v2.	3
Sensitive-data protection	Automatic secret and personal-data detection with a redaction queue, more role levels (e.g. Guest), and a full audit log of views, approvals and exports.	3

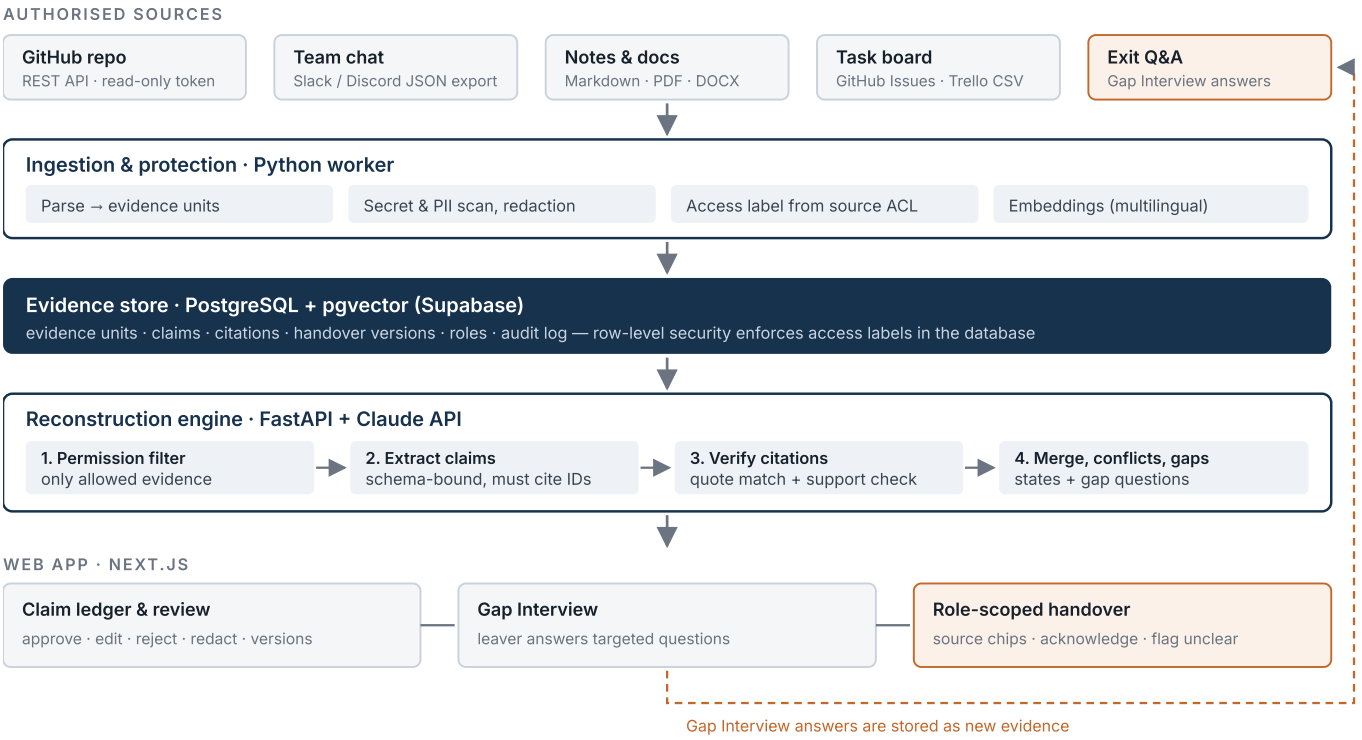
Export	Approved handover exported as PDF/Markdown with citations kept as links.	3

Deliberately out of scope during the event

- **Live Slack/Discord/Notion OAuth sync.** Exports are official, permission-respecting and enough to prove the workflow. Live sync is future scope.
- **Meeting audio and transcription.** Text notes only.
- **Mobile app, Slack bot, HR integrations.** Not needed to complete a handover.
- **Real organisational data.** All demos use synthetic or public data, as the brief requires.

§7 · TECHNICAL PLAN

Architecture, stack and evidence model



Technology stack and why we chose it

Layer	Choice	Why this, for this problem
Frontend	Next.js (TypeScript), Tailwind CSS, shadcn/ui	Fast to build review-heavy interfaces (tables, side panels, diff views); easy deployment; one language for UI work.
Backend API	Python, FastAPI	Best ecosystem for parsing, text processing and AI; typed request/response models with Pydantic double as the claim schema.
Database & auth	PostgreSQL + pgvector on Supabase, Supabase Auth	One service gives relational data, vector search and auth. **Row-level security** lets us enforce access labels inside the database, not just in the UI — the main reason we picked it.
AI model	Anthropic Claude API	Strong at long, messy inputs and at following strict output schemas (tool use / structured JSON), which our citation-bound extraction depends on.
Embeddings	multilingual-e5-small (open source, runs locally)	Free, private, and handles the mixed Hindi–English (“Hinglish”) chat that is normal in Indian student teams.
Background jobs	Python worker + Postgres job table	Ingestion takes minutes; a job table avoids running a separate queue service.
Supporting libraries	PyGithub, pypdf, python-docx, rapidfuzz, detect-secrets	Proven open-source tools for source parsing, quote matching and secret detection.
Hosting	Vercel (web), Render (API + worker), Supabase	Free tiers are sufficient for judging; judges get a live link.

Alternatives considered: a separate vector database (Pinecone, Chroma) — rejected because pgvector keeps permissions and vectors in one place; LangChain-style agents — rejected in favour of a small, explicit pipeline we can test step by step.

Evidence model

The data model is what makes traceability and permissions enforceable rather than promised.

Entity	Key fields	Purpose
Source	`type, name, scope, default_access_label, connected_by`	One connected repo, export or folder.
EvidenceUnit	`source_id, author, timestamp, text, url/locator, access_label, redactions[], embedding`	The smallest citable piece. Every claim must point to these.
Claim	`type, statement, state, confidence_reason, workstream, owner`	Decision, owner, risk, dependency, next action, open question, contact.
Citation	`claim_id, evidence_id, quote_span, support (full / partial)`	Links a claim to the exact text that supports it.
Handover	`leaver, successor, successor_role, status, version, approved_by, published_at`	One handover with locked, versioned snapshots.
ReviewAction / Ack	`who, action, before/after, time · section, acknowledged / unclear`	Audit trail of human control and successor receipt.

Integrations and data sources

Source	How we access it	Why it is realistic during the event
GitHub	REST API with a fine-grained, read-only personal access token for one repository	The repo owner controls the token scope; works for public and private repos.
Slack / Discord	Official workspace export (Slack) or channel export (Discord JSON), uploaded by an admin	No app approval or OAuth review needed; the admin decides which channels to export.
Notes / docs	File upload (MD, PDF, DOCX) or Google Docs exported as files	Covers meeting notes, specs and existing READMEs.
Task board	GitHub Issues/Projects via API, Trello board CSV/JSON export	Most student teams use one of the two.
Demo data	Synthetic project “CampusKart”: a public repo we create during the event plus a generated chat export and notes	Satisfies the brief’s synthetic-data rule and lets us plant known facts for testing (§11).

The hard parts, and how we handle them

Challenge 1 — Making every claim trustworthy (the main one)

Language models produce confident text that may not be supported. A handover that invents a reason for a decision is worse than no handover. We do not rely on prompting alone; we wrap the model in checks we control.

#	Step	How it works
1	**Bounded input**	The model sees one batch at a time (a thread or a time window), with every evidence unit tagged by a stable ID.
2	**Schema-bound extraction**	Output must be JSON matching our claim schema: type, statement, cited evidence IDs and the exact quote that supports it. Free text is rejected.
3	**Deterministic checks**	Every cited ID must exist and must have been in that batch; the quoted span must fuzzy-match the source text (rapidfuzz). Failures are dropped or downgraded to `INFERRED`.
4	**Support check**	A second, narrow model call sees only the claim and its cited evidence and answers: supports / partially / does not. Only "supports" can become `VERIFIED`.
5	**Human confirmation**	Nothing is published until a reviewer approves it; reviewer edits are stored as `CONFIRMED` with name and time.

Challenge 2 — Not leaking what the successor shouldn't see

If the model reads a private channel while writing a contributor's handover, the information can surface in a summary even if the channel is "hidden" in the UI. So we filter **before** generation:

- Each evidence unit inherits an access label from its source (e.g. `#core-team → leads`, `finance/ → restricted`).
- A handover view is generated only from units the target role may read. Postgres row-level security enforces this at query level, so a bug in the app layer cannot bypass it.
- A claim takes the most restrictive label of its evidence. If the successor cannot see all of its evidence, they do not see the claim.
- Secrets (API keys, tokens) and personal data are detected and redacted at ingestion; reviewers can redact any span manually before publishing.

Challenge 3 — Contradictory and stale information

Sources disagree all the time. Claims about the same subject are grouped using embeddings and an entity key (module, service, person); each group is checked for contradiction. Conflicts are shown with both sides and their dates — the newest source is *not* assumed to be correct. Publishing is blocked until each conflict is resolved or explicitly marked unresolved.

Challenge 4 — Long, noisy input and cost

A semester of chat can be thousands of messages of jokes, memes and off-topic talk. We filter obvious noise cheaply (bots, reactions, very short messages), thread messages before extraction, cache results per batch so re-runs only process new evidence, and cap batch size so cost stays predictable on student credits. If the AI API fails or rate-limits, jobs retry with back-off and the UI shows which sources are incomplete instead of producing a partial handover silently.

How we meet the brief's trust requirements

Requirement in the brief	How Continuum meets it
Do not present generated assumptions as verified organisational truth.	Five explicit evidence states; inferences labelled in the text itself; only checked claims can be "Verified"; reviewer confirmation is recorded separately from AI verification.
Do not expose confidential information merely because it appeared in a connected source.	Source-level scoping at connect time, access labels, permission filtering before generation, row-level security, secret/PII redaction, reviewer "mark sensitive".
Allow authorised users to correct, reject and approve the final handover.	Claim-level approve / edit / reject / redact, version history, publish lock, and an audit log of every action.
Use synthetic, public, permissioned or anonymised data for demonstrations.	The demo runs on our synthetic "CampusKart" project only. The product asks the admin to confirm they are authorised to connect each source.

Data handling

- Read-only access to every source; Continuum never writes back to GitHub or chat.
- Only the excerpts needed for a batch are sent to the AI API, after redaction.
- Tokens and uploaded files are stored encrypted and scoped to one workspace.
- Deleting a handover deletes its evidence units, embeddings and files.
- The leaver can see what the system extracted about their work before it is shared.
- Accessibility: keyboard navigation and text labels on every status — colour is never the only signal.

> **Honest limitations we will state in the product**
>
> Continuum can only reconstruct what was written down somewhere. Knowledge that lives purely in someone's head reaches the handover only through the Gap Interview, and a reviewer still has to judge whether the answers are right. We would rather show a clear "Missing" than fill the gap with a plausible guess.

§10 · TEAM RESPONSIBILITIES

Who builds what

We are a two-person team, so the split is clean: one of us owns the data and AI pipeline, the other owns everything the user touches. We share the demo dataset, testing and documentation, and both of us will be able to explain the whole system.

TEAM LEAD · BACKEND & AI

Pawan Singh Bhandari

Personally builds

- Source connectors: GitHub API, Slack/Discord export and document parsers
- Evidence store schema, access labels and row-level security policies
- Claim extraction pipeline with the Claude API and the claim schema
- Verification layer: citation checks, support check, evidence states
- Conflict and gap detection; Gap Interview question generator
- FastAPI endpoints, background worker, backend deployment

Also owns

Architecture decisions, AI cost and reliability, technical sections of the round documents.

FRONTEND & PRODUCT

Rahul Bramhankar

Personally builds

- Next.js app: workspace setup, create-handover and connect-sources screens
- Claim ledger review interface: approve / edit / reject / redact, readiness score, version history
- Evidence viewer: click a source chip to see the original text in context
- Gap Interview page for the leaver
- Successor "Start here" handover view with acknowledgement and "unclear" flags
- Auth and role-based routing on the frontend; web deployment

Also owns

User interviews, usability testing, demo flow and the final presentation.

Shared work

Task	Split
Synthetic "CampusKart" dataset (repo history, chat export, notes) with planted facts for testing	Pawan: repo and planted facts · Rahul: chat and notes
Test plan, end-to-end testing, failure cases	Pawan: pipeline tests · Rahul: user journeys
Round 2 and Round 3 documentation, README and setup guide	Both, reviewed by each other

§11 · DEVELOPMENT PLAN ACROSS THREE ROUNDS

A plan we can actually finish

Round	Goal	Deliverable
1 · The Plan 5–6 Nov	Understand the problem, define users, design a trustworthy workflow and a realistic scope.	This proposal.
2 · The Build 7–10 Nov · 72 h	The full central workflow working end to end on synthetic data: Connect → Reconstruct → Verify → Review → Handover.	Deployed app, public repo, architecture document, test evidence, three committed improvements.
3 · The Finish 13–16 Nov · 72 h	Deliver the three improvements, harden reliability and permissions, polish usability, document limitations.	Final product, demo, report, testing notes, privacy and accessibility explanation.

Round 2 build milestones

Round 2 is a 72-hour build (7–10 November, from 10:00). Hours are counted from the start of the window.

Milestone	Owner	Hours 0–18	18–36	36–54	54–72
M1 · Repo, schema, auth, demo dataset	Both				
M2 · Connectors → evidence units	Pawan				
M3 · Extraction + verification + gaps	Pawan				
M4 · Review UI, evidence viewer, handover view	Rahul				
M5 · Integrate, deploy, test, document	Both				

Rule we will follow: by hour 36 the end-to-end flow is connected with simple versions of every stage, **before** any stage is polished, so a broken central step is never discovered at the end. Before the window opens we will only plan and research, as the rules allow; no code is written outside the window.

Three improvements we plan to commit to for Round 3

1. Cross-source conflict resolution

Project-wide contradiction detection with a side-by-side resolution screen.

2. Acknowledgement loop + version diff

Successor acknowledges or flags sections; flags route back; reviewers see v1 → v2 changes.

3. Sensitive-data protection

Automatic secret/PII detection, redaction queue, extra roles and a full audit log.

If we fall behind — the cut list

- Drop the Trello connector first (GitHub Issues stays).
- Keep DOCX out; support Markdown and PDF notes only.
- Use one role view instead of two, keeping the permission filter itself.
- **Never cut:** citations, evidence states, review and approval — they are the product.

How we will test it

- **Planted-facts test:** the demo project contains known decisions, conflicts and missing owners. We measure how many are found and how many shown claims carry a valid citation.
- **Permission test:** generate a contributor view and check that no text from lead-only sources appears.
- **Failure cases:** empty sources, malformed exports, API timeouts.

§12 · EXPECTED IMPACT

What changes for each person

User	Before	With Continuum
Leaver	Writes a long handover document in their last week, from memory.	Answers a short list of targeted questions; the rest is reconstructed from work already done.
Successor	Spends the first weeks reverse-engineering decisions and finding who to ask.	Starts from a “Start here” page with a first-week plan, risks and contacts — and can check every line against its source.
Reviewer	Has no way to check the handover or control what is shared.	Reviews claim by claim, sees exactly what each role will see, and keeps an audit trail.
Team / club	Loses momentum every semester; the same mistakes repeat.	Keeps a growing, verified decision log that survives each change of people.

How we will measure it

We will not claim numbers we have not measured. In Rounds 2 and 3 we will report: share of displayed claims with a valid citation (target: all of them), share of planted facts recovered, number of gap questions per handover, reviewer edit rate, and time for a test successor to answer a fixed set of “can you continue this project?” questions with and without Continuum.

§13 · FUTURE SCOPE

After the event

Always-on handover readiness

Run continuously, not only at departure. Show each team which modules depend on a single person (the “bus factor”) long before anyone leaves.

Live connectors

OAuth sync for Slack, Discord, Notion, Google Drive, Jira and Linear, respecting each tool’s own permissions automatically.

Successor Q&A

Let the successor ask questions in plain language, answered only from approved, permitted evidence with citations.

Self-hosted and local-model option

For organisations that cannot send data to an external API: on-premise deployment with an open-weight model.

Templates by setting

Handover templates for research groups (datasets, experiments), NGOs (donors, volunteers) and startups (customers, infra).

Club and college programme

Offer Continuum to college tech clubs as an end-of-semester handover tool — our first real users and a path to adoption.

§14 · TEAM AND PARTICIPANT DETAILS

Team Binary Builders

Team name	**Binary Builders**
Event	CodeFrontier Hackathon (We Code Coders) — Round 1: The Plan
Problem statement	01 — Prevent Critical Knowledge From Leaving With People
Product	Continuum — evidence-backed project handovers
Team size	2

#	Participant	Role	Part they will personally build
1	**Pawan Singh Bhandari** Team lead	Backend · AI / ML	Source connectors, evidence store with row-level security, claim extraction and verification pipeline, conflict and gap detection, Gap Interview generator, API and worker.
2	**Rahul Bramhankar**	Frontend · Design / UI-UX	Web app: connect-sources flow, claim ledger review and approval, evidence viewer, Gap Interview page, successor handover view with acknowledgement, user testing.

Disclosures

Item	Disclosure
Prior work	No code, template or prototype for this product exists. All development will happen in the official build window, in a public repository with full commit history.
Third-party services	Anthropic Claude API (claim extraction and support checks), Supabase (database, auth), Vercel and Render (hosting), GitHub REST API.
Open-source components	Next.js, Tailwind CSS, shadcn/ui, FastAPI, Pydantic, pgvector, multilingual-e5-small, PyGithub, pypdf, python-docx, rapidfuzz, detect-secrets — used under their licences.
AI assistance	This proposal was drafted with AI assistance (Anthropic Claude) and reviewed and approved by both team members. AI coding assistants may also be used during development; all such use will be listed in the repository README.
Data	Demonstrations use only synthetic data created by the team. No real organisational or personal data will be used.

In one sentence: Continuum helps a leaving team member hand over a project through a short set of targeted answers instead of a long document written from memory, and gives the successor a handover they can trust — because every line shows where it came from, what is uncertain, and that a human approved it.